PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)
PROJECT REPORT
TOPIC- MONGODB STUDENT DATABASE OPERATIONS

SUBMITTED TO- MR. Harendra Singh Rajpoot
Submitted by- Zeeshan Ahmad
BCADS27
Roll no. - 1250258518
3rd Semester
Babu Banarasi Das University
Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not. The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 51 students, and demonstrates the standard database operations — insertion, updation, deletion and querying — that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document into the students collection.

Code:

```
db.students.insertOne({
rollNo: 201,
name: "Aisha Khan",
age: 20,
department: "CSE",
marks: 88
})
```

Output:

```
{
acknowledged: true,
insertedId: ObjectId('...')
}
```

2. Insert Many

Inserts multiple student documents (roll numbers 202 to 251) into the collection at the same time.

Code:

```
db.students.insertMany([
{ rollNo: 202, name: "Raunak Joshi", age: 19, department: "IT", marks: 76 },
{ rollNo: 203, name: "Deepti Sharma", age: 21, department: "CSE", marks: 93 },
{ rollNo: 204, name: "Farhan Ali", age: 20, department: "ECE", marks: 81 },
{ rollNo: 205, name: "Tanvi Mehta", age: 22, department: "ME", marks: 74 },
{ rollNo: 206, name: "Siddharth Rao", age: 19, department: "CSE", marks: 89 },
{ rollNo: 207, name: "Pooja Verma", age: 20, department: "IT", marks: 69 },
{ rollNo: 208, name: "Nikhil Gupta", age: 21, department: "ECE", marks: 95 },
{ rollNo: 209, name: "Lavanya Nair", age: 20, department: "CSE", marks: 83 },
{ rollNo: 210, name: "Aryan Sinha", age: 19, department: "IT", marks: 77 },
{ rollNo: 211, name: "Neeta Patil", age: 22, department: "ME", marks: 72 },
{ rollNo: 212, name: "Kabir Das", age: 20, department: "CSE", marks: 90 },
{ rollNo: 213, name: "Swati Tiwari", age: 21, department: "ECE", marks: 67 },
{ rollNo: 214, name: "Harish Kumar", age: 19, department: "IT", marks: 91 },
{ rollNo: 215, name: "Megha Saxena", age: 20, department: "CSE", marks: 79 },
{ rollNo: 216, name: "Ritesh Pandey", age: 21, department: "ECE", marks: 84 },
{ rollNo: 217, name: "Anita Bajaj", age: 22, department: "ME", marks: 73 },
{ rollNo: 218, name: "Saurabh Mishra", age: 20, department: "CSE", marks: 96
},
{ rollNo: 219, name: "Priti Dubey", age: 19, department: "IT", marks: 68 },
{ rollNo: 220, name: "Rajesh Yadav", age: 21, department: "ECE", marks: 87 },
```

```
{ rollNo: 221, name: "Nidhi Agarwal", age: 20, department: "CSE", marks: 75 },
{ rollNo: 222, name: "Vipin Chauhan", age: 19, department: "IT", marks: 90 },
{ rollNo: 223, name: "Kaveri Iyer", age: 22, department: "ME", marks: 71 },
{ rollNo: 224, name: "Sudhir Pillai", age: 20, department: "CSE", marks: 86 },
{ rollNo: 225, name: "Namita Ghosh", age: 21, department: "ECE", marks: 80 },
{ rollNo: 226, name: "Lokesh Reddy", age: 19, department: "IT", marks: 94 },
{ rollNo: 227, name: "Chitra Bose", age: 20, department: "CSE", marks: 77 },
{ rollNo: 228, name: "Manav Jain", age: 21, department: "ECE", marks: 82 },
{ rollNo: 229, name: "Divyesh Shah", age: 22, department: "ME", marks: 65 },
{ rollNo: 230, name: "Trupthi Nair", age: 20, department: "CSE", marks: 93 },
{ rollNo: 231, name: "Omkar Deshpande", age: 19, department: "IT", marks: 70
},
{ rollNo: 232, name: "Sarika Kulkarni", age: 21, department: "ECE", marks: 89
},
{ rollNo: 233, name: "Piyush Tripathi", age: 20, department: "CSE", marks: 81
},
{ rollNo: 234, name: "Rashmi Shukla", age: 19, department: "IT", marks: 85 },
{ rollNo: 235, name: "Gaurav Bhatt", age: 22, department: "ME", marks: 78 },
{ rollNo: 236, name: "Komal Srivastava", age: 20, department: "CSE", marks: 97
},
{ rollNo: 237, name: "Aditya Menon", age: 21, department: "ECE", marks: 74 },
{ rollNo: 238, name: "Pallavi Dube", age: 19, department: "IT", marks: 88 },
{ rollNo: 239, name: "Hemant Joshi", age: 20, department: "CSE", marks: 66 },
{ rollNo: 240, name: "Shruti Kapoor", age: 21, department: "ECE", marks: 92 },
{ rollNo: 241, name: "Pranav Tomar", age: 22, department: "ME", marks: 69 },
{ rollNo: 242, name: "Nisha Bhardwaj", age: 20, department: "CSE", marks: 84
},
{ rollNo: 243, name: "Rohan Malhotra", age: 19, department: "IT", marks: 79 },
{ rollNo: 244, name: "Varsha Patel", age: 21, department: "ECE", marks: 91 },
{ rollNo: 245, name: "Sandeep Rathore", age: 20, department: "CSE", marks: 87
},
{ rollNo: 246, name: "Meena Thakur", age: 19, department: "IT", marks: 73 },
{ rollNo: 247, name: "Kiran Dixit", age: 21, department: "ECE", marks: 86 },
{ rollNo: 248, name: "Sunil Rawat", age: 22, department: "ME", marks: 68 },
{ rollNo: 249, name: "Anjali Singh", age: 20, department: "CSE", marks: 90 },
{ rollNo: 250, name: "Navneet Kaur", age: 21, department: "IT", marks: 76 },
{ rollNo: 251, name: "Yashika Goel", age: 19, department: "ECE", marks: 83 }
])
```

3. Update One

Updates the first document that matches the given condition — here, the marks of roll number 201 are updated from 88 to 92.

Code:

```
db.students.updateOne(
{ rollNo: 201 },
{ $set: { marks: 92 } }
)
```

Output:

```
{
acknowledged: true,
matchedCount: 1,
modifiedCount: 1
}
```

4. Update Many

Updates all documents that match the given condition — here, every student in the CSE department is given a new status field set to "Active".

Code:

```
db.students.updateMany(
{ department: "CSE" },
{ $set: { status: "Active" } }
)
```

Output:

```
{
acknowledged: true,
matchedCount: 17,
modifiedCount: 17
}
```

5. Delete One

Deletes the first document that matches the given condition — here, the record of roll number 251 is removed.

Code:

```
db.students.deleteOne({ rollNo: 251 })
```

Output:

```
{
acknowledged: true,
deletedCount: 1
}
```

6. Delete Many

Deletes all documents that match the given condition — here, every student with marks less than 40 is removed.

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:

```
{
acknowledged: true,
deletedCount: 0
}
```

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records.

Marks greater than 80:

Code:

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

```
201 Aisha Khan 20 CSE 88
203 Deepti Sharma 21 CSE 93
204 Farhan Ali 20 ECE 81
206 Siddharth Rao 19 CSE 89
208 Nikhil Gupta 21 ECE 95
209 Lavanya Nair 20 CSE 83
212 Kabir Das 20 CSE 90
214 Harish Kumar 19 IT 91
216 Ritesh Pandey 21 ECE 84
218 Saurabh Mishra 20 CSE 96
220 Rajesh Yadav 21 ECE 87
222 Vipin Chauhan 19 IT 90
224 Sudhir Pillai 20 CSE 86
226 Lokesh Reddy 19 IT 94
228 Manav Jain 21 ECE 82
230 Trupthi Nair 20 CSE 93
232 Sarika Kulkarni 21 ECE 89
233 Piyush Tripathi 20 CSE 81
234 Rashmi Shukla 19 IT 85
236 Komal Srivastava 20 CSE 97
238 Pallavi Dube 19 IT 88
240 Shruti Kapoor 21 ECE 92
242 Nisha Bhardwaj 20 CSE 84
244 Varsha Patel 21 ECE 91
245 Sandeep Rathore 20 CSE 87
247 Kiran Dixit 21 ECE 86
249 Anjali Singh 20 CSE 90
251 Yashika Goel 19 ECE 83
```

Marks less than 80:

Code:

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
202 Raunak Joshi 19 IT 76
205 Tanvi Mehta 22 ME 74
207 Pooja Verma 20 IT 69
210 Aryan Sinha 19 IT 77
211 Neeta Patil 22 ME 72
213 Swati Tiwari 21 ECE 67
215 Megha Saxena 20 CSE 79
217 Anita Bajaj 22 ME 73
219 Priti Dubey 19 IT 68
221 Nidhi Agarwal 20 CSE 75
223 Kaveri Iyer 22 ME 71
227 Chitra Bose 20 CSE 77
229 Divyesh Shah 22 ME 65
231 Omkar Deshpande 19 IT 70
235 Gaurav Bhatt 22 ME 78
237 Aditya Menon 21 ECE 74
239 Hemant Joshi 20 CSE 66
241 Pranav Tomar 22 ME 69
243 Rohan Malhotra 19 IT 79
246 Meena Thakur 19 IT 73
248 Sunil Rawat 22 ME 68
250 Navneet Kaur 21 IT 76
```

Marks equal to 85:

Code:

```
db.students.find({ marks: { $eq: 85 } })
```

Output:

```
234 Rashmi Shukla 19 IT 85
```

Marks greater than or equal to 80:

Code:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

```
201 Aisha Khan 20 CSE 88
203 Deepti Sharma 21 CSE 93
204 Farhan Ali 20 ECE 81
206 Siddharth Rao 19 CSE 89
208 Nikhil Gupta 21 ECE 95
209 Lavanya Nair 20 CSE 83
212 Kabir Das 20 CSE 90
214 Harish Kumar 19 IT 91
216 Ritesh Pandey 21 ECE 84
218 Saurabh Mishra 20 CSE 96
220 Rajesh Yadav 21 ECE 87
222 Vipin Chauhan 19 IT 90
224 Sudhir Pillai 20 CSE 86
225 Namita Ghosh 21 ECE 80
226 Lokesh Reddy 19 IT 94
228 Manav Jain 21 ECE 82
230 Trupthi Nair 20 CSE 93
232 Sarika Kulkarni 21 ECE 89
233 Piyush Tripathi 20 CSE 81
234 Rashmi Shukla 19 IT 85
236 Komal Srivastava 20 CSE 97
238 Pallavi Dube 19 IT 88
240 Shruti Kapoor 21 ECE 92
242 Nisha Bhardwaj 20 CSE 84
244 Varsha Patel 21 ECE 91
245 Sandeep Rathore 20 CSE 87
247 Kiran Dixit 21 ECE 86
249 Anjali Singh 20 CSE 90
251 Yashika Goel 19 ECE 83
```

Marks not equal to 85:

Code:

```
db.students.find({ marks: { $ne: 85 } })
```

```
201 Aisha Khan 20 CSE 88
202 Raunak Joshi 19 IT 76
203 Deepti Sharma 21 CSE 93
204 Farhan Ali 20 ECE 81
205 Tanvi Mehta 22 ME 74
206 Siddharth Rao 19 CSE 89
207 Pooja Verma 20 IT 69
208 Nikhil Gupta 21 ECE 95
209 Lavanya Nair 20 CSE 83
210 Aryan Sinha 19 IT 77
211 Neeta Patil 22 ME 72
212 Kabir Das 20 CSE 90
213 Swati Tiwari 21 ECE 67
214 Harish Kumar 19 IT 91
215 Megha Saxena 20 CSE 79
216 Ritesh Pandey 21 ECE 84
217 Anita Bajaj 22 ME 73
218 Saurabh Mishra 20 CSE 96
219 Priti Dubey 19 IT 68
220 Rajesh Yadav 21 ECE 87
221 Nidhi Agarwal 20 CSE 75
222 Vipin Chauhan 19 IT 90
223 Kaveri Iyer 22 ME 71
224 Sudhir Pillai 20 CSE 86
225 Namita Ghosh 21 ECE 80
... (all students except those with marks = 85)
```

Marks greater than 70 AND age less than 23:

Code:

```
db.students.find({
$and: [
{ marks: { $gt: 70 } },
{ age: { $lt: 23 } }
]
})
```

Output:

```
201 Aisha Khan 20 CSE 88
202 Raunak Joshi 19 IT 76
203 Deepti Sharma 21 CSE 93
204 Farhan Ali 20 ECE 81
205 Tanvi Mehta 22 ME 74
206 Siddharth Rao 19 CSE 89
208 Nikhil Gupta 21 ECE 95
209 Lavanya Nair 20 CSE 83
210 Aryan Sinha 19 IT 77
211 Neeta Patil 22 ME 72
212 Kabir Das 20 CSE 90
214 Harish Kumar 19 IT 91
215 Megha Saxena 20 CSE 79
216 Ritesh Pandey 21 ECE 84
217 Anita Bajaj 22 ME 73
218 Saurabh Mishra 20 CSE 96
220 Rajesh Yadav 21 ECE 87
221 Nidhi Agarwal 20 CSE 75
222 Vipin Chauhan 19 IT 90
223 Kaveri Iyer 22 ME 71
224 Sudhir Pillai 20 CSE 86
225 Namita Ghosh 21 ECE 80
226 Lokesh Reddy 19 IT 94
227 Chitra Bose 20 CSE 77
228 Manav Jain 21 ECE 82
230 Trupthi Nair 20 CSE 93
232 Sarika Kulkarni 21 ECE 89
233 Piyush Tripathi 20 CSE 81
234 Rashmi Shukla 19 IT 85
235 Gaurav Bhatt 22 ME 78
236 Komal Srivastava 20 CSE 97
237 Aditya Menon 21 ECE 74
238 Pallavi Dube 19 IT 88
240 Shruti Kapoor 21 ECE 92
242 Nisha Bhardwaj 20 CSE 84
243 Rohan Malhotra 19 IT 79
244 Varsha Patel 21 ECE 91
245 Sandeep Rathore 20 CSE 87
246 Meena Thakur 19 IT 73
247 Kiran Dixit 21 ECE 86
249 Anjali Singh 20 CSE 90
250 Navneet Kaur 21 IT 76
```

```
251 Yashika Goel 19 ECE 83
```

Department is CSE OR ECE:

Code:

```
db.students.find({
$or: [
{ department: "CSE" },
{ department: "ECE" }
]
})
```

Output:

```
[
  { _id: ObjectId('...'), rollNo: 201, name: 'Aisha Khan', age: 20, department: 'CSE', marks: 88, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 203, name: 'Deepti Sharma', age: 21, department: 'CSE', marks: 93, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 204, name: 'Farhan Ali', age: 20, department: 'ECE', marks: 81 },
  { _id: ObjectId('...'), rollNo: 206, name: 'Siddharth Rao', age: 19, department: 'CSE', marks: 89, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 208, name: 'Nikhil Gupta', age: 21, department: 'ECE', marks: 95 },
  { _id: ObjectId('...'), rollNo: 209, name: 'Lavanya Nair', age: 20, department: 'CSE', marks: 83, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 212, name: 'Kabir Das', age: 20, department: 'CSE', marks: 90, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 213, name: 'Swati Tiwari', age: 21, department: 'ECE', marks: 67 },
  { _id: ObjectId('...'), rollNo: 215, name: 'Megha Saxena', age: 20, department: 'CSE', marks: 79, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 216, name: 'Ritesh Pandey', age: 21, department: 'ECE', marks: 84 },
  { _id: ObjectId('...'), rollNo: 218, name: 'Saurabh Mishra', age: 20, department: 'CSE', marks: 96, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 220, name: 'Rajesh Yadav', age: 21, department: 'ECE', marks: 87 },
  { _id: ObjectId('...'), rollNo: 221, name: 'Nidhi Agarwal', age: 20, department: 'CSE', marks: 75, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 224, name: 'Sudhir Pillai', age: 20, department: 'CSE', marks: 86, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 225, name: 'Namita Ghosh', age: 21, department: 'ECE', marks: 80 },
  { _id: ObjectId('...'), rollNo: 227, name: 'Chitra Bose', age: 20, department: 'CSE', marks: 77, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 228, name: 'Manav Jain', age: 21, department: 'ECE', marks: 82 },
  { _id: ObjectId('...'), rollNo: 230, name: 'Trupthi Nair', age: 20, department: 'CSE', marks: 93, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 232, name: 'Sarika Kulkarni', age: 21, department: 'ECE', marks: 89 },
  { _id: ObjectId('...'), rollNo: 233, name: 'Piyush Tripathi', age: 20, department: 'CSE', marks: 81, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 236, name: 'Komal Srivastava', age: 20, department: 'CSE', marks: 97, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 237, name: 'Aditya Menon', age: 21, department: 'ECE', marks: 74 },
  { _id: ObjectId('...'), rollNo: 239, name: 'Hemant Joshi', age: 20, department: 'CSE', marks: 66, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 240, name: 'Shruti Kapoor', age: 21, department: 'ECE', marks: 92 },
  { _id: ObjectId('...'), rollNo: 242, name: 'Nisha Bhardwaj', age: 20, department: 'CSE', marks: 84, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 244, name: 'Varsha Patel', age: 21, department: 'ECE', marks: 91 },
  { _id: ObjectId('...'), rollNo: 245, name: 'Sandeep Rathore', age: 20, department: 'CSE', marks: 87, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 247, name: 'Kiran Dixit', age: 21, department: 'ECE', marks: 86 },
  { _id: ObjectId('...'), rollNo: 249, name: 'Anjali Singh', age: 20, department: 'CSE', marks: 90, status: 'Active' },
  { _id: ObjectId('...'), rollNo: 251, name: 'Yashika Goel', age: 19, department: 'ECE', marks: 83 }
]
```

Marks NOT greater than 80:

Code:

```
db.students.find({
marks: { $not: { $gt: 80 } }
})
```

Output:

```
202 Raunak Joshi 19 IT 76
205 Tanvi Mehta 22 ME 74
207 Pooja Verma 20 IT 69
210 Aryan Sinha 19 IT 77
211 Neeta Patil 22 ME 72
213 Swati Tiwari 21 ECE 67
215 Megha Saxena 20 CSE 79
217 Anita Bajaj 22 ME 73
219 Priti Dubey 19 IT 68
221 Nidhi Agarwal 20 CSE 75
223 Kaveri Iyer 22 ME 71
225 Namita Ghosh 21 ECE 80
227 Chitra Bose 20 CSE 77
229 Divyesh Shah 22 ME 65
231 Omkar Deshpande 19 IT 70
235 Gaurav Bhatt 22 ME 78
237 Aditya Menon 21 ECE 74
239 Hemant Joshi 20 CSE 66
241 Pranav Tomar 22 ME 69
243 Rohan Malhotra 19 IT 79
246 Meena Thakur 19 IT 73
248 Sunil Rawat 22 ME 68
250 Navneet Kaur 21 IT 76
```

8. MongoDB Operator Summary

Operator	Meaning	Example
`$gt`	Greater than	marks: { $gt: 80 }
`$lt`	Less than	marks: { $lt: 80 }
`$eq`	Equal to	marks: { $eq: 85 }
`$gte`	Greater than or equal to	marks: { $gte: 80 }
`$ne`	Not equal to	marks: { $ne: 85 }
`$and`	All conditions must be true	marks > 70 AND age < 23
`$or`	At least one condition must be true	CSE OR ECE
`$not`	Negates a condition	NOT (marks > 80)

Total dataset: 51 students (Roll No. 201–251).

Implementation Details

• The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).

• A single collection named "students" is used, with fields: rollNo, name, age, department and marks.

• insertOne() and insertMany() are used to populate the collection with 51 student records.

• updateOne() and updateMany() with the $set operator are used to modify existing fields and add new ones (e.g. status).

• deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.

• find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and, $or, $not) to retrieve specific subsets of student data.

• Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 51 student records. The updateOne operation successfully changed one student's marks, while updateMany correctly modified all 17 CSE-department documents to add an "Active" status. The deleteOne operation removed one record and deleteMany found no records matching its filter, correctly reporting a deletedCount of 0. The query operations correctly filtered students by marks and department using comparison and logical operators, demonstrating that MongoDB's query engine returns accurate, condition-based results across a realistic-sized dataset.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

• Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.

• Add schema validation rules to enforce data types and required fields at the database level.

• Create indexes on frequently queried fields (e.g. marks, department) to improve performance.

• Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.

• Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

• MongoDB Official Documentation: https://www.mongodb.com/docs/

• MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/

• MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/

• mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/

THANK YOU!